Evaluación de Unidad

1. Datos Generales

Asignatura	Álgebra Lineal
Ciclo	1 "A"
Unidad	1
Resultado de aprendizaje de la unidad	Aplica y resuelve analíticamente utilizando diversos métodos sistemas de ecuaciones lineales, matrices y determinantes, aplicaciones lineales según diversos criterios y utilizando diversas técnicas para la resolución de problemas con ayuda de software matemático, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad
Nombre del Docente	Edwin René Guamán Quinche
Nombre del Estudiante	Alison Nagua Cruz
Fecha	04 al 10 de noviembre de 2026
Horario	7h30 - 9h30
Link de github	https://github.com/alisonnagua/-lgebra_lineal.git

2. Objetivo:

- Demostrar la capacidad de identificar, modelar matemáticamente y resolver un problema práctico utilizando el álgebra lineal aplicando sistemas de ecuaciones lineales o matrices.

3. Contexto

En este proyecto se analiza el juego tres en raya en el que el tablero se representa como una matriz de 3x3 donde las fichas X y O y las casillas vacías se codifican con los valores de 1, -1 y 0. Con esta representación para detectar un ganador se calculan sumas de filas, columnas y diagonales mediante productos matriz-vector y trazas [1], y girar o reflejar el tablero se expresa con una matriz de permutación. Se eligió este tema porque es un juego conocido y permite ver con claridad cómo una situación cotidiana puede modelarse matemáticamente.

Este proyecto es relevante en algebra lineal porque representa cambios mediante matrices, es una técnica básica en áreas como el procesamiento de imágenes y los gráficos por computador donde rotar, reflejar o transformar datos es algo habitual. Con este proyecto se aprenderá a modelar un problema real con matrices, a resolver las operaciones paso a paso de forma manual y a implementarlas en Python comparando los resultados del cálculo a mano con los de la aplicación.

4. Desarrollo de las operaciones

4.1. Planteamiento del problema

Dado el tablero final de una partida de tres en raya, representado como una matriz, determinar si hay un ganador y cómo se ve el tablero al rotarlo 90°.

Codificación: X = 1, O = -1 y casilla vacía = 0.
Matriz A:

A = [[1, -1, 1], [0, 1, -1], [-1, 0, 1]]
Matrices auxiliares:

u = [[1], [1], [1]] (vector de unos)

J = [[0, 0, 1], [0, 1, 0], [1, 0, 0]] (matriz antidiagonal)

Operación a realizar:

1. *Suma de filas: A * u*
2. *Suma de columnas: u^T * A*
3. *Traza de A (suma de la diagonal principal)*
4. *Suma de la diagonal secundaria: traza de A * J*
5. *Decidir el ganador: si alguna suma vale 3 gana X, si vale -3 gana O*
6. Rotar el tablero 90° (A^T * J)

4.2. Procedimiento paso a paso

1. Paso 1: *Suma de filas: A * u*

Es una multiplicación de la matriz A por el vector de unos u, como el vector de u contiene solo unos la multiplicación equivale a sumar los tres elementos de cada fila de la matriz A

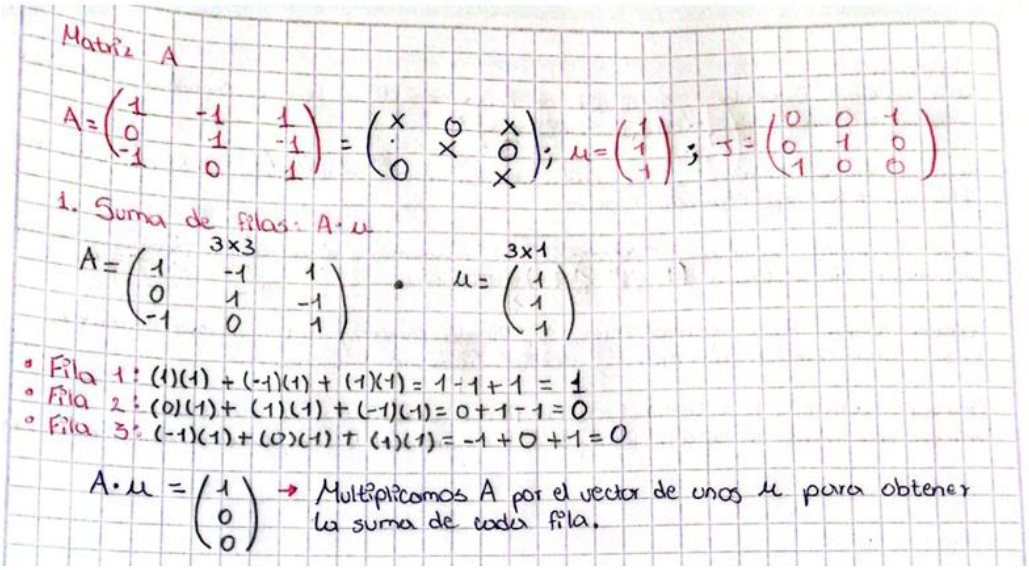


Fig. 1. Suma de filas A * u

Ninguna suma es 3 ni -3 lo que significa que no hay línea ganadora en las filas

2. Paso 2: *Suma de columnas: u^T * A*

Es una multiplicación de el vector de unos u transpuesto por la matriz de A, como el vector u contiene solo unos la multiplicación equivale a sumar los tres elementos de cada columna de la matriz A

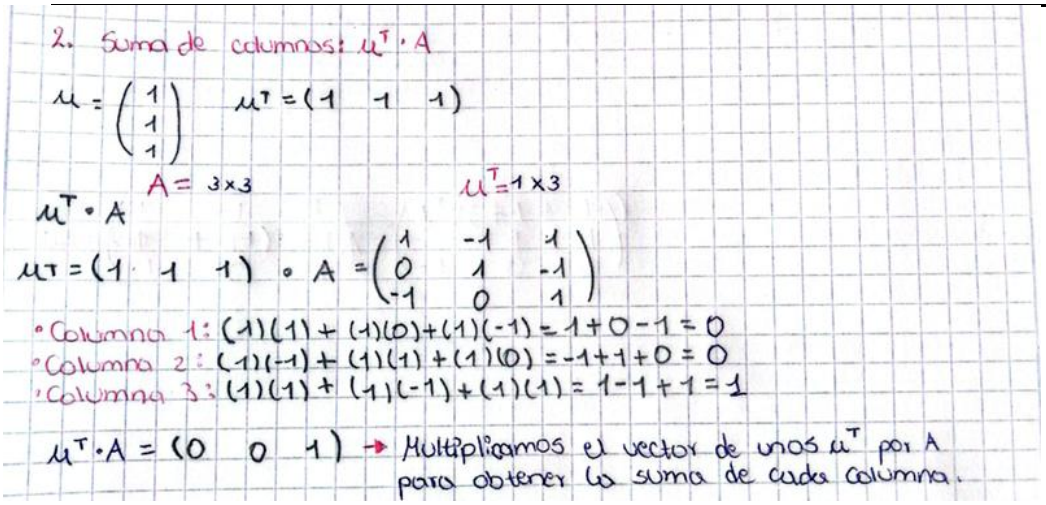


Fig. 2. Suma de columnas $u^T \cdot A$

Ninguna suma es 3 ni -3 lo que significa que no hay línea ganadora en las columnas

3. Paso 3: La traza de una matriz es la suma de los números de su diagonal principal, la que va de arriba-izquierda abajo-derecha.

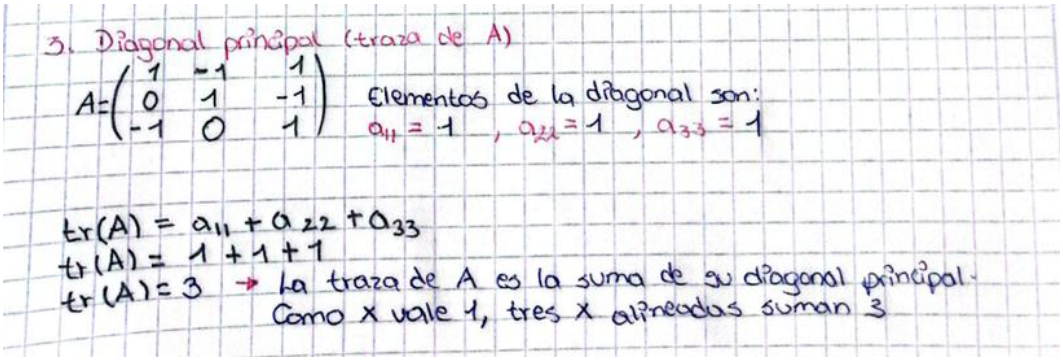


Fig. 3. Diagonal principal (traza de A)

El resultado es 3 entonces la diagonal principal de X es la ganadora

4. Paso 4: Diagonal secundaria (traza de A * J)

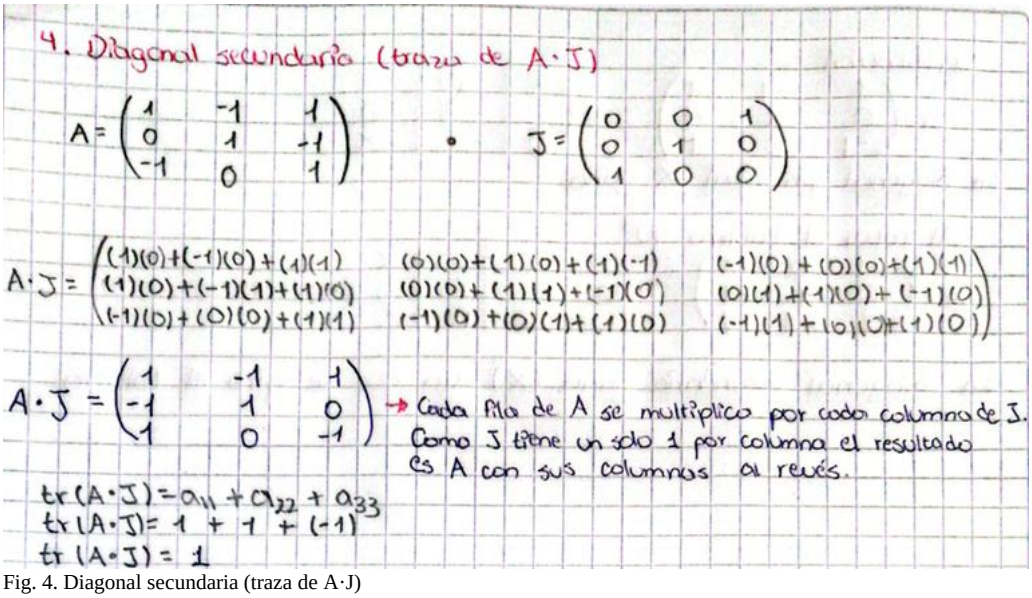


Fig. 4. Diagonal secundaria (traza de A·J)

Se multiplica A por J para invertir el orden de las columnas, así la diagonal secundaria queda como diagonal principal. Su traza es 1 no es 3 ni -3 significa que esta diagonal no es ganadora

5. Paso 5: Decidir el ganador*: si alguna suma vale 3 gana X, si vale -3 gana O*

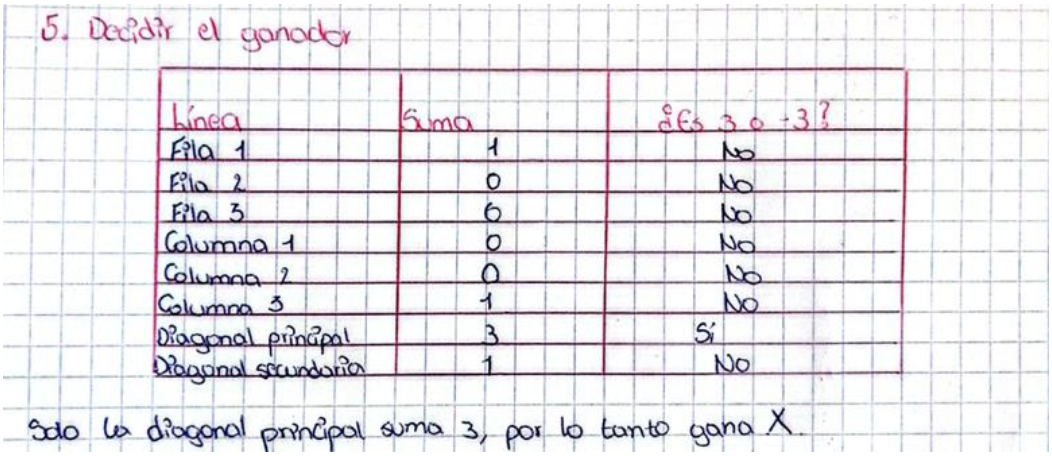


Fig. 5. Decidir el ganador

Se revisa las 8 líneas posibles del tablero (3 filas, 2 columnas y 2 diagonales). Como X vale 1, tres X alineadas suman 3. La diagonal principal suma 3 y ninguna otra línea suma 3 o -3, es por eso que gana X

6. Paso 6: Rotar el tablero 90° ($A^T * J$)

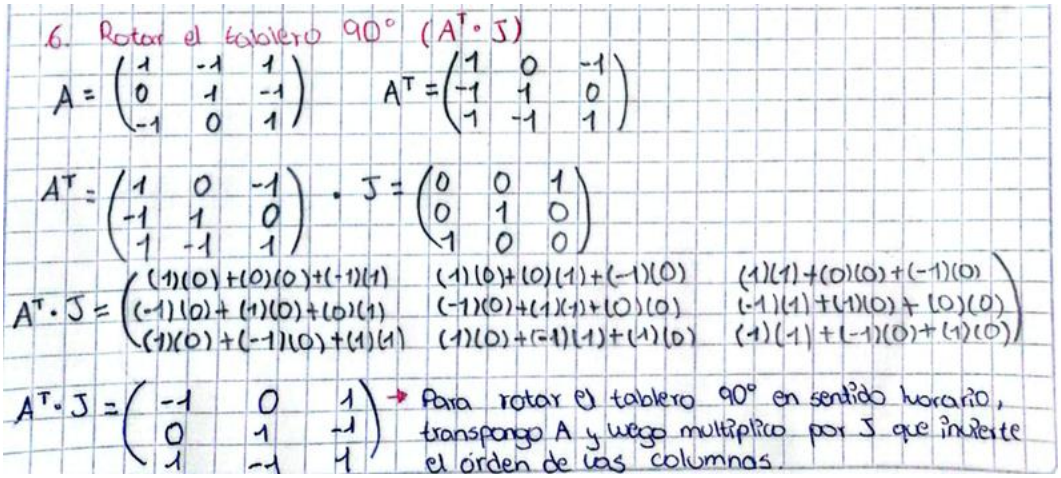


Fig. 6. Rotar el tablero 90°

Esta es la matriz al rotar el tablero 90° en sentido horario, la transpuesta intercambia filas por columnas y el producto por la matriz J invierte el orden de las columnas [1].

4.3 Verificación de resultados

1. Verificar que la diagonal principal sea la ganadora

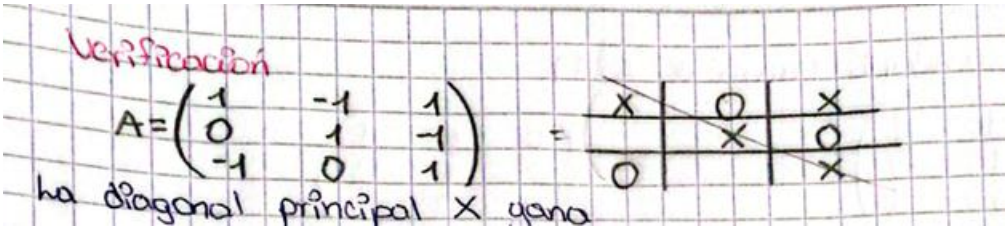


Fig. 7. Verificar la diagonal ganadora

2. Verificar el tablero a 90° y la diagonal ganadora

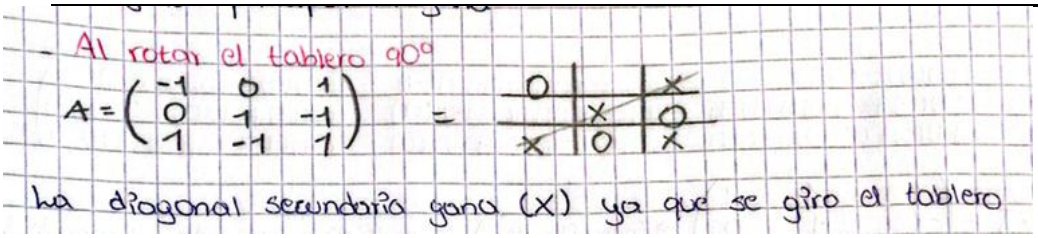


Fig. 8. Verificar el tablero a 90°

5. Procedimiento / Metodología

5.1 Descripción de la aplicación

La aplicación es un programa en Python que representa un tablero de tres en raya como una matriz de 3x3 y determina el ganador mediante operaciones algebraicas. Está dirigida a estudiantes que quieren ver cómo un problema cotidiano se forma con matrices. Su objetivo principal es verificar por computadora los cálculos hechos a mano en la sección 4 de este documento, esto se relaciona con algebra lineal ya que usa productos matriz-vector, la traza, la transpuesta y el producto de matrices.

5.2 Descripción de la aplicación

Categoría	Herramienta / Tecnología	Versión / Detalle
Lenguaje de programación	Python	3.13.18
Entorno de desarrollo (IDE)	Google Colab	Notebook en la nube
Librerías / Frameworks	NumPy	2.4.6
Plataforma de ejecución	Web	Navegador

5.3 Funcionalidades implementadas

- Funcionalidad 1: **Representar el tablero como matriz**. El tablero se guarda en un arreglo de NumPy de 3x3 (A), donde X = 1, O = -1 y la casilla vacía = 0. — Aplica: matrices
- Funcionalidad 2: **Calcular las sumas de filas y columnas.** Se multiplica A por el vector de unos (A @ u) y el vector de unos por A (u @ A) para obtener la suma de cada fila y de cada columna — Aplica: producto matriz-vector
- Funcionalidad 3: **Calcular las diagonales**. Se usa np.trace(A) para la diagonal principal y np.trace(A @ J) para la diagonal secundaria — Aplica: traza de una matriz y matriz antidiagonal J
- Funcionalidad 4: **Detectar el ganador.** Se revisan todas las sumas y si alguna suma vale 3 gana X y si vale -3 gana O — Aplica: uso de los resultados de cada operación para decidir el ganador
- Funcionalidad 5: **Rotar el tablero 90°.** Se calcula $A^T \cdot J$ en la que se transpone A y se invierte el orden de las columnas con J.— Aplica: matriz transpuesta y producto de matrices

5.4 Capturas de pantalla

- Código de Python que detecta el ganador y rota el tablero con las operaciones de algebraicas

```python
import numpy as np

# ---------- Matrices base ----------
A = np.array([[ 1, -1,  1],
              [ 0,  1, -1],
              [-1,  0,  1]])           # tablero: X = 1, O = -1, vacío = 0
u = np.ones(3, dtype=int)              # vector de unos
J = np.fliplr(np.eye(3, dtype=int))    # matriz antidiagonal

# ---------- Para mostrar X y O ----------
SIMBOLOS = {1: "X", -1: "O", 0: " "}

def mostrar_tablero(M):
    for i in range(3):
        print(" " + " | ".join(SIMBOLOS[v] for v in M[i]))
        if i < 2:
            print("---+---+---")

print("Tablero original:")
mostrar_tablero(A)
print("\nA =\n", A)

# Paso 1: suma de filas (A · u)
print("\nPaso 1 - Suma de filas (A·u):", A @ u)

# Paso 2: suma de columnas (uᵀ · A; NumPy trata u como fila)
print("Paso 2 - Suma de columnas (uᵀ·A):", u @ A)

# Paso 3: diagonal principal (traza de A)
print("Paso 3 - Diagonal principal, tr(A):", np.trace(A))

# Paso 4: diagonal secundaria (traza de A·J)
print("Paso 4 - Diagonal secundaria, tr(A·J):", np.trace(A @ J))
print("   A·J =\n", A @ J)

# Paso 5: ganador
sumas = np.concatenate([A @ u, u @ A, [np.trace(A), np.trace(A @ J)]])
if 3 in sumas:
    print("Paso 5 - Ganador: X")
elif -3 in sumas:
    print("Paso 5 - Ganador: O")
else:
    print("Paso 5 - No hay ganador")

# Paso 6: rotación 90° (Aᵀ · J)
R = A.T @ J
print("\nPaso 6 - Aᵀ =\n", A.T)
print("Paso 6 - R = Aᵀ·J =\n", R)

print("\nTablero rotado 90°:")
mostrar_tablero(R)
```

Fig. 9. Código en Google Colab

- Salida del código para el tablero A: gana X y se muestra la matriz rotada

```
Tablero original:
 X | O | X
---+---+---
   | X | O
---+---+---
 O |   | X

A =
 [[ 1 -1  1]
 [ 0  1 -1]
 [-1  0  1]]

Paso 1 - Suma de filas (A·u): [1 0 0]
Paso 2 - Suma de columnas (u^T·A): [0 0 1]
Paso 3 - Diagonal principal, tr(A): 3
Paso 4 - Diagonal secundaria, tr(A·J): 1
   A·J =
 [[ 1 -1  1]
 [-1  1  0]
 [ 1  0 -1]]
Paso 5 - Ganador: X

Paso 6 - A^T =
 [[ 1  0 -1]
 [-1  1  0]
 [ 1 -1  1]]
Paso 6 - R = A^T·J =
 [[-1  0  1]
 [ 0  1 -1]
 [ 1 -1  1]]

Tablero rotado 90°:
 O |   | X
---+---+---
   | X | O
---+---+---
 X | O | X
```

Fig. 10. Salida del código en Google Colab

5.5 Código fuente (fragmentos clave)

- **Bloque 1:** matrices base (A, u y J)

```
// import numpy as np

A = np.array([[ 1, -1,  1],
              [ 0,  1, -1],
              [-1,  0,  1]])            # tablero: X = 1, O = -1,
vacío = 0
u = np.ones(3, dtype=int)               # vector de unos
J = np.fliplr(np.eye(3, dtype=int))     # matriz antidiagonal
```

- **Bloque 2: Cálculos de las matrices**

```
// # Paso 1: suma de filas (A · u)
print("Paso 1:", A @ u)

# Paso 2: suma de columnas (uᵀ · A)
print("Paso 2:", u @ A)

# Paso 3: diagonal principal (traza de A)
print("Paso 3:", np.trace(A))

# Paso 4: diagonal secundaria (traza de A · J)
print("Paso 4:", np.trace(A @ J))

# Paso 6: rotación 90° (Aᵀ · J)
print("Paso 6:\n", A.T @ J)
```

- **Bloque 3: Decidir el ganador**

```
// sumas = np.concatenate([A @ u, u @ A, [np.trace(A), np.trace(A @ J)]])

if 3 in sumas:
    print("Paso 5 - Ganador: X")
elif -3 in sumas:
    print("Paso 5 - Ganador: O")
else:
    print("Paso 5 - No hay ganador")
```

Repositorio / Enlace al proyecto:
https://colab.research.google.com/drive/1YKHsfwkxegLSfQhS_KRL7BnuHTWip2cY

6. Resultados

Reflexiona sobre los resultados obtenidos tanto en la parte manual como en la aplicación computacional. Responde las siguientes preguntas orientadoras:

- ¿Los resultados del cálculo manual coinciden con los de la aplicación? ¿Qué diferencias existen y a qué se deben?

Los resultados del cálculo manual coincidieron con la aplicación, el ganador coincide con X en la diagonal principal (traza = 3) y la matriz rotada coincide con la que realizo en el paso 6, no existió diferencias con los cálculos esperados

- ¿Cuáles fueron las mayores dificultades que encontraste al resolver el problema manualmente? ¿Y al programarlo?

Al realizar los cálculos se presentaron dificultades debido a que el método de filas por columnas resultaba confuso en ocasiones, también ocurrieron errores en la aplicación de los signos al momento de calcular las matrices 3x3 y me existió complejidad para comprender porque multiplicar por J invertía las columnas.

Al momento de programar lo más complicado fue comprender el código generado por Gemini debido a que no lograba entender cómo la IA pretendía que funcionara el programa.

- ¿En qué situaciones reales de la computación o ingeniería se aplica el tema que estudiaste?

En la representación de estados mediante matrices se aplica en áreas de la computación y la ingeniería como en el procesamiento de imágenes cuando una imagen que es una matriz de píxeles y operaciones empieza a rotar o al reflejarla se calculan con transposiciones y matrices de permutación [1]. como la rotación $A^T \cdot J$ usada en el proyecto. También se aplica en los gráficos por computador donde rotar, reflejar y transformar objetos se hace mediante productos de matrices, así es como el tres en raya usa las mismas herramientas que se usan en aplicaciones mucho más grandes

- ¿Qué aprendiste al construir la aplicación que no hubieras aprendido solo con teoría?

Al construir la aplicación aprendí que cada paso del cálculo manual tiene una línea equivalente en el código. Por ejemplo, la suma de filas A * u se escribe A @ u y la rotación $A^T \cdot J$ se escribe A.T @ J lo que me ayudó a entender mejor qué cada operación. También aprendí a verificar el resultado con ayuda de Gemini en Python, comparé mis resultados a mano y comprobé que el ganador X y la matriz rotada coincidían. Además, noté detalles que la teoría no muestra, como que NumPy trata el vector u como fila cuando va a la izquierda de la matriz, por lo que no hace falta escribir la transpuesta

7. Conclusiones

- Al codificar X como 1, O = -1 y la casilla vacía como 0, detectar un ganador se reduce a comprobar si alguna suma de fila, columna o diagonal vale 3 o -3. En la matriz A la traza dio 3 y tanto el cálculo manual como la salida del programa confirmaron que gana X usando productos matriz-vector y trazas
- La rotación del tablero se expresa con una sola operación $A^T \cdot J$ que combina la transpuesta y el producto por una matriz de permutación [1]. El resultado del paso 6 hecho a mano coincidió con el calculado de Python
- Cada línea del código corresponde a un paso del cálculo manual (por ejemplo, A @ u es la suma de filas) por lo que el programa sirvió para verificar el procedimiento hecho a mano. NumPy permite expresar estas operaciones de matrices de forma directa, sin recorrer el tablero casilla por casilla [2].

8. Referencias

[1] D. C. Lay, S. R. Lay y J. J. McDonald, *Álgebra lineal y sus aplicaciones*, 5.ª ed. México, D. F., México: Pearson Educación, 2016.

[2] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.